



BSc and MPhys Degree Assessments 2023/24

School:

PHYSICS, ENGINEERING AND TECHNOLOGY

Course:

SCIENTIFIC SUPERCOMPUTING — ASSIGNMENT

Allocation of Marks:

This assignment is marked out of 100.

Instructions:

Answer ALL sections.

This is an open assignment. You may refer to notes, textbooks and online information.

You may not discuss this assignment, through direct or indirect means, with any student or any other person until the results and feedback have been released.

Work should be submitted as single archive file in ZIP or tar format – other formats (e.g. tar, RAR, 7zip etc) are NOT acceptable. This archive is to contain a document (either Word DOC with embedded fonts or PDF format – other formats e.g. ODT are NOT acceptable) that contains all parts of your solution, including a listing of all your programs, AND separate files containing all the source code, header files, scripts etc. **Failure to comply with these requirements will incur a 10% mark penalty.**

You must ensure that no identifying information (including your exam number) appears anywhere within your submission. This includes file names, program comments, PDF properties, etc.

Please submit your solution via the Scientific Supercomputing VLE page.

You may not discuss this assignment, through direct or indirect means, with any student or any other person until the results and feedback have been released.

If – and only if – the VLE will not accept your PDF, you may e-mail it to phys-emergency-vle-submissions@york.ac.uk, ensuring that the subject line of your e-mail and the name of the attached file consist of the module code, and the assignment number.

The Table of Constants and Mathematical Formulae are on the departmental Wiki [here](#).

Turn over

Answer ALL questions.

- 1 ARCHER2 <https://www.archer2.ac.uk> is the current UK national supercomputer. Bede <https://n8cir.org.uk/docs/bede/> is the current N8 tier-2 supercomputer. These two machines have very different architectures (see the websites for details and also the Bede documentation). Compare and contrast the different hardware, and comment on the design decisions that went into each machine. Which do you think would be best for a biomolecular simulation using Gromacs and why?

[2 sides A4 max]

[15]

- 2 Consider a simple model 2D problem, which has a regular 2D square grid of data with periodic boundary conditions on all edges. Using the same approach and assumptions as in the lectures, perform your own “cost of parallel decomposition analysis”, and hence derive expressions for the figure of merit, $P\Sigma/N_{grid}$, for both 1D and 2D data decompositions. How many processors are required before a 2D decomposition scheme becomes more efficient?

[10]

- 3 Download this tar-gzip file from http://www-users.york.ac.uk/~mijp1/exam_q3.tar.gz and unpack it using

```
gunzip exam_q3.tar.gz
tar -xf exam_q3.tar
```

and then move into the new directory created. This contains a simple CFD code, in both Fortran and C++ variants. Decide which language variant you wish to work with, and then compile it using appropriate flags. When executed, the code will produce some output to the screen, and also create an output file called `shock_bubble_fort_serial.out` or `shock_bubble_cpp_serial.out` as appropriate.

- (a) Profile the code and identify the performance bottleneck(s). Explain why you think these sections of the code are slow, and what can be done to accelerate them using OpenMP. Include the appropriate section(s) of the profile output in your answer.

[5]

- (b) Add appropriate OpenMP directives and function calls to improve the performance of the code. Explore different OpenMP directives to achieve the best parallel speedup you can get. Explain what you have done and why, and what actions (if any) were necessary to ensure the code gives the same answer on any number of threads (both output to screen and output to file) as in serial.

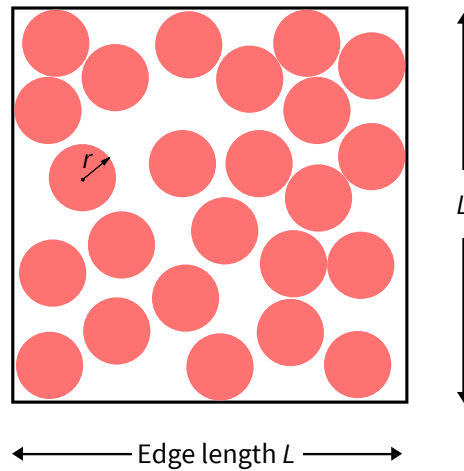
[15]

- (c) What is the scaling behaviour of your modified code on G Viking? What speedup can you get with `OMP_NUM_THREADS` varying from 1 to 48? Include a graph of

speedup in your answer. Explain the behaviour you observe in terms of the code and the hardware.

[5]

- 4 Random packing is often an issue for packaging of goods to correctly estimate the volume or weight of a product within a box. The packing fraction depends on the shapes of the contents and packaging and for random packing the solution is not analytical, requiring a computational model of the packing of objects. Such a model can also be used to approximate the structure of porous materials such as sponges. The packing fraction is often optimised by shaking to reduce the size of voids, but the first stage is to randomly pack the contents in a starting configuration. Consider random 2D packing of uniform circles of radius r with a 2D box with edge length L :



The initial packing is done by generating random x and y coordinates for each circle and attempting to place the circle in the box so that it does not overlap the box boundary or any other circle in the box. This is determined by ensuring the distance between the centre of any two circles is greater than $2r$. New circles are generated until no more circles can be placed, requiring many attempts as the number of possible voids decreases. The final packing fraction P is given by:

$$P = \frac{\pi N_{\text{circ}} r^2}{L^2}$$

where N_{circ} is the number of circles placed in the box, r is the radius of the circles, and L is the size of the box.

- (a) Write a serial computer program to calculate the maximum packing fraction P as defined in the equation above for a box size $L = 500$ and circle radius $r = 1.234$. Show the convergence of the packing fraction P as a function of the number of trial circle placements and give your final value of P in your answer. The specific answer you obtain will depend on your choice of random number generator and seed. Show a segment of your box with randomly placed circles bounded by the square $(0, 0.025L)$ in each direction demonstrating that the circles do not overlap

each other or the box edges. Marks will be awarded for correctness and efficiency [10], neatness/clarity of coding [5] and comments [5].

[20]

- (b) Modify your serial program to run in parallel using an appropriate parallelization strategy and method (eg OpenMP, MPI, CUDA, OpenCL, OpenACC). Justify your chosen parallelization strategy and method. Demonstrate how your program scales with different numbers of processes/threads and with problem size by considering different ratios r/L comparing both **strong** and **weak** scaling. Include appropriate graphs and clearly document all relevant parameters, including which machine(s) you have used, compilers and flags. The scaling tests should be conducted on HPC hardware such as GViking, Viking or similar. Marks will be awarded on the basis of correct parallelization of the serial code [10], efficiency [5] and the scalability [15] of the code.

[30]

End of paper